Twelfth Implementation Iteration: Space Allocation

with Abstract Request Base Class

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this twelfth implementation iteration is to improve the object-oriented design of the prototype by introducing an abstract base class for requests. In previous iterations, `OneTimeRequest` and `RecurringRequest` were separate classes with duplicated shared attributes.

In this iteration, a new abstract class `Request` is introduced. This improves code reuse and enables polymorphic rule evaluation for the shared request properties.

2. Changes from Iteration 11

Compared with the eleventh iteration, the following changes were introduced:

- Added a new abstract base class: `Request`
- Moved shared request attributes into `Request`
- Refactored `OneTimeRequest` and `RecurringRequest` to inherit from `Request`
- Updated `CapacityRule`, `FeatureRule`, `StatusRule`, and `LocationRule` to operate on the base `Request` type.

3. Scenario

A user submits either a one-time request or a recurring request. Both types now inherit common structure from the abstract `Request` class. The system evaluates shared properties polymorphically through the base type.

4. Modules and Their Tasks

A. Request Management Module

- Represent shared request data in the abstract `Request` base class.
- Provide polymorphic access to common request properties.

B. Resource Management Module

- Manage all supported space types: `Classroom`, `Laboratory`, and `MeetingRoom`.

C. Rule Evaluation Module

- Apply rules (`Capacity`, `Feature`, `Status`, `Location`) on the base `Request` type.

5. Iteration Comparison

Iter.	**Requests**	**Resources**	**Rules Added**
1–4	OTR Only	Class → Lab → Meet	Availability + Cap.
5–6	OTR + RR	Class + Lab + Meet	(Redesign)
7	OTR + RR	Class + Lab + Meet	+ Feature
8–9	OTR + RR	Class + Lab + Meet	+ Status
10–11	OTR + RR	Class + Lab + Meet	+ Location
12	**Abs. Request**	Class + Lab + Meet	**Polymorphism**

Note: OTR = OneTimeRequest, RR = RecurringRequest. Rules are cumulative.

6. Updated Class Diagram

Figure 1 shows the updated simplified class diagram showing the new Inheritance structure.

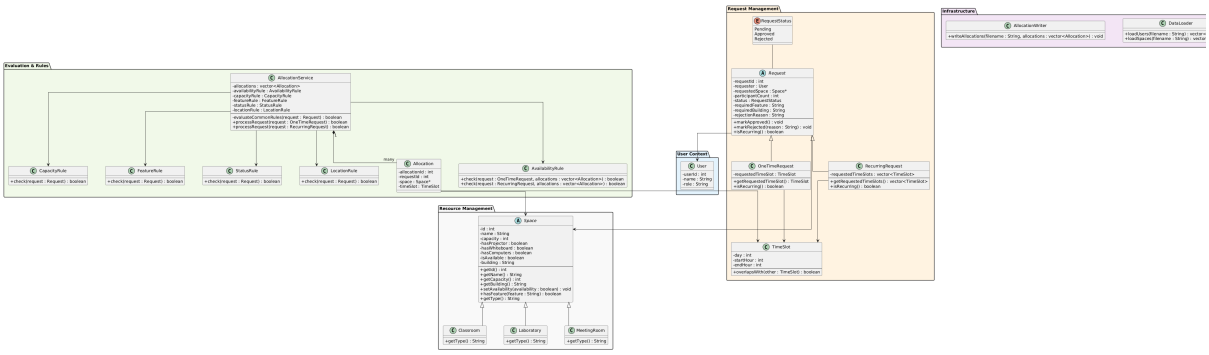

Figure 1: Updated Simplified Class Diagram

7. Updated Workflow Diagram

Figure 2 shows the workflow for the twelfth iteration. The internal logic now uses polymorphic calls.

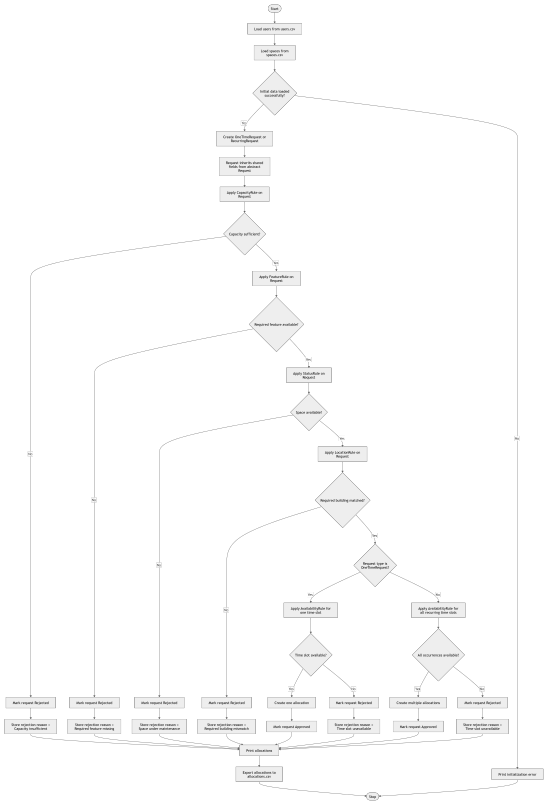

Figure 2: Updated Workflow Diagram

8. Prototype Implementation

The C++ prototype was refactored. Shared rule classes were updated to operate polymorphically on the base request type, and the allocation service was refactored to reuse a shared helper method for common rule checks.

9. Summary

This iteration improves the internal object-oriented design without changing visible behavior. Future iterations can extend this polymorphism to the Resource model.